

Thuật toán II

Hoàng Ngọc Long

Ngày 27 tháng 10 năm 2025

Bài tập 1:

a. Chứng minh tính đúng đắn

1. Cơ sở quy nạp:

Với $n = 0$:

- Theo định nghĩa $a^0 = 1$
- Thuật toán trả về 1.
- $\Rightarrow$ Thuật toán đúng với $n = 0$

2. Giả thiết quy nạp:

Giả sử thuật toán **đúng** với một số nguyên không âm k , tức là:

$$power(a, k) = a^k$$

3. Bước quy nạp:

Ta cần chứng minh thuật toán cũng **đúng** với $n = k + 1$ (với $k \geq 0$). Ta xem xét cách thuật toán tính $power(a, k + 1)$:

1. Vì $k + 1 > 0$, thuật toán áp dụng trường hợp đệ quy:

$$power(a, k + 1) = a \times power(a, (k + 1) - 1)$$

$$power(a, k + 1) = a \times power(a, k)$$

2. Áp dụng Giả thiết quy nạp ($power(a, k) = a^k$):

$$power(a, k + 1) = a \times a^k$$

3. Áp dụng định nghĩa và tính chất của lũy thừa ($a^1 \times a^k = a^{1+k}$):

$$power(a, k + 1) = a^{k+1}$$

$\Rightarrow$ Thuật toán đúng với $n = k + 1$.

$\Rightarrow$ Điều phải chứng minh

b. Xây dựng hệ thức truy hồi và đánh giá thời gian chạy

1. Xây dựng hệ thức truy hồi

Gọi $T(n)$ là thời gian (số phép toán cơ bản) để tính a^n bằng thuật toán đệ quy cơ bản. Thuật toán đệ quy được định nghĩa lại:

$$Power(a, n) = \begin{cases} 1 & \text{nếu } n = 0 \\ a \cdot Power(a, n - 1) & \text{nếu } n > 0 \end{cases}$$

Trường hợp cơ sở ($n = 0$)

Khi $n = 0$, thuật toán thực hiện một phép trả về hằng số 1.

$$T(0) = c_1$$

(với c_1 là hằng số)

Trường hợp đệ quy ($n > 0$)

Khi $n > 0$, thuật toán thực hiện:

1. Một lần gọi đệ quy để tính $Power(a, n - 1)$. Thời gian chạy là $T(n - 1)$.
2. Một phép nhân ($a \cdot Power(a, n - 1)$).
3. Một số phép toán cơ bản khác (so sánh $n > 0$, trả về).

Ta có thể biểu diễn thời gian chạy như sau:

$$T(n) = T(n - 1) + c_2$$

(với c_2 là hằng số, đại diện cho thời gian của phép nhân và các phép toán cơ bản khác)

Hệ thức truy hồi hoàn chỉnh

Hệ thức truy hồi cho thuật toán tính a^n là:

$$\begin{cases} T(n) = T(n - 1) + c_2 & \text{nếu } n > 0 \\ T(0) = c_1 \end{cases}$$

2. Đánh giá thời gian chạy

Để tìm độ phức tạp thời gian, ta giải hệ thức truy hồi $T(n) = T(n - 1) + c_2$. Ta mở rộng hệ thức truy hồi (phương pháp lặp):

$$T(n) = T(n - 1) + c_2$$

$$T(n) = (T(n - 2) + c_2) + c_2 = T(n - 2) + 2c_2$$

$$T(n) = (T(n - 3) + c_2) + 2c_2 = T(n - 3) + 3c_2$$

...

Tiếp tục lặp cho đến khi đạt đến trường hợp cơ sở $T(0)$ (tức là sau $k = n$ lần lặp):

$$T(n) = T(n - k) + k \cdot c_2$$

Đặt $k = n$:

$$T(n) = T(n - n) + n \cdot c_2$$

$$T(n) = T(0) + n \cdot c_2$$

Thay $T(0) = c_1$:

$$T(n) = c_1 + c_2 n$$

Vì $T(n)$ là một hàm tuyến tính của n , với c_1 và c_2 là các hằng số, ta có độ phức tạp thời gian tiệm cận:

$$T(n) \in O(n)$$

Kết luận Đánh giá Thời gian chạy

Độ phức tạp thời gian chạy của thuật toán đệ quy cơ bản tính a^n là $O(n)$ (tuyến tính). Điều này là do thuật toán thực hiện chính xác n phép nhân để tính a^n , và thời gian chạy tỉ lệ trực tiếp với số mũ n .

Bài tập 2:

a. Chứng minh tính đúng đắn

Ta chứng minh tính đúng đắn bằng phương pháp **Quy nạp Toán học** trên **chiều dài của đoạn con đang xét**, $L = j - i + 1$.

Cơ sở quy nạp

Xét $L = 1$, tức là $i = j$. Đoạn con chỉ có một phần tử duy nhất: $A[i]$.

1. Nếu $A[i] = x$:

- Lệnh `if  $a_i = x$  then return  $i$` được thực thi.
- Thuật toán trả về i , là chỉ số đúng của x . (Đúng)

2. Nếu $A[i] \neq x$:

- Lệnh `if  $a_i = x$  then bị bỏ qua.`
- Lệnh `else if  $i = j$  then return 0` được thực thi (vì $i = j$).
- Thuật toán trả về 0, báo hiệu không tìm thấy. (Đúng)

$\Rightarrow$ Thuật toán đúng đắn với chiều dài $L = 1$.

Giả thiết quy nạp

Giả sử thuật toán **đúng đắn** với mọi đoạn con có chiều dài $L = k$ (tức là $j - i + 1 = k$).

Bước quy nạp

Xét đoạn con có chiều dài $L = k + 1$.

1. Nếu $A[i] = x$ (phần tử đầu tiên):

- Lệnh `if  $a_i = x$  then return  $i$` được thực thi.
- Thuật toán trả về i . (Đúng)

2. Nếu $A[i] \neq x$ (phần tử đầu tiên):

- Lệnh `if  $i = j$  then` bị bỏ qua vì $L = k + 1 > 1$, nên $i \neq j$.
- Thuật toán đi vào nhánh `else return LinearSearch( $i + 1, j, x$ )`.
- Đây là lời gọi đệ quy trên đoạn con $A[i + 1..j]$. Chiều dài của đoạn con mới là $j - (i + 1) + 1 = j - i = (k + 1) - 1 = k$.
- Theo **Giả thiết Quy nạp**, $LinearSearch(i + 1, j, x)$ hoạt động đúng đắn trên đoạn con $A[i + 1..j]$. Nó sẽ trả về chỉ số đúng $k \in \{i + 1, \dots, j\}$ nếu tìm thấy x , hoặc trả về 0 nếu không tìm thấy x trong đoạn $A[i + 1..j]$.
- Vì x không phải là $A[i]$, việc tìm kiếm trong $A[i + 1..j]$ là hoàn toàn chính xác để xác định sự có mặt của x trong $A[i..j]$.

$\Rightarrow$ Thuật toán đúng đắn với chiều dài $L = k + 1$.

Theo Nguyên lý Quy nạp Toán học, thuật toán $LinearSearch(i, j, x)$ là đúng đắn với mọi đoạn con $A[i..j]$ (điều phải chứng minh).

b. Hệ thức truy hồi và đánh giá thời gian chạy

Ta xét thời gian chạy trong trường hợp xấu nhất (Worst-Case), tức là x không có trong đoạn con $A[i..j]$ hoặc $x = A[j]$.

Gọi $T(L)$ là thời gian chạy của thuật toán trên một đoạn con có chiều dài $L = j - i + 1$.

A. Xây dựng hệ thức truy hồi

1. Trường hợp cơ sở ($L = 1$):

- Khi $i = j$ (chiều dài $L = 1$), thuật toán thực hiện 1 phép so sánh ($a_i = x$), 1 phép so sánh ($i = j$), và các thao tác return.

$$T(1) = c_1$$

(với c_1 là hằng số)

2. Trường hợp đệ quy ($L > 1$):

- Khi $i \neq j$, thuật toán thực hiện:
 - 1 phép so sánh ($a_i = x$).
 - 1 phép so sánh ($i = j$).

- 1 lần gọi đệ quy $LinearSearch(i + 1, j, x)$. Chiều dài của đoạn con mới là $L - 1$. Thời gian là $T(L - 1)$.

$$T(L) = T(L - 1) + c_2$$

(với c_2 là hằng số, đại diện cho thời gian của 2 phép so sánh và các thao tác khác)

Hệ thức truy hồi hoàn chỉnh:

$$\begin{cases} T(L) = T(L - 1) + c_2 & \text{nếu } L > 1 \\ T(1) = c_1 \end{cases}$$

Đánh giá thời gian chạy

Giải hệ thức truy hồi bằng phương pháp lặp:

$$T(L) = T(L - 1) + c_2$$

$$T(L) = (T(L - 2) + c_2) + c_2 = T(L - 2) + 2c_2$$

$\vdots$

$$T(L) = T(L - k) + k \cdot c_2$$

Ta lặp cho đến khi $L - k = 1$, tức là $k = L - 1$:

$$T(L) = T(L - (L - 1)) + (L - 1)c_2$$

$$T(L) = T(1) + (L - 1)c_2$$

Thay $T(1) = c_1$:

$$T(L) = c_1 + (L - 1)c_2$$

$$T(L) = c_1 + c_2L - c_2 = c_2L + (c_1 - c_2)$$

Vì $T(L)$ là một hàm tuyến tính của L , ta có độ phức tạp thời gian tiệm cận:

$$T(L) \in \mathbf{O(L)}$$

Nếu đoạn con ban đầu là toàn bộ mảng ($i = 1, j = n$), thì $L = n$.

Kết luận: Độ phức tạp thời gian chạy của thuật toán Tìm kiếm Tuyến tính đệ quy (trường hợp xấu nhất) là $O(n)$, trong đó n là chiều dài của đoạn con đang xét.

Bài tập 7:

a. $T(n) = 4T(n/3) + n^2$

- $a = 4, b = 3, f(n) = n^2$.
- $f(n) \in O(n^d) \implies d = 2$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_3 4}$ và n^2 .
- Ta có $\log_3 4 \approx 1.26 < 2$.
- $\implies n^{\log_3 4} \in O(n^2)$ và n^2 lớn hơn $n^{\log_3 4}$ theo một yếu tố đa thức ($n^2 \in \Omega(n^{\log_3 4 + \epsilon})$ với $\epsilon \approx 0.74$).
- Theo Định lý Thợ, Trường hợp 3:

$$T(n) = \mathbf{O}(n^2)$$

b. $T(n) = 4T(n/2) + n^2$

- $a = 4, b = 2, f(n) = n^2$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_2 4} = n^2$ và n^2 .
- Ta có $n^{\log_b a} = f(n)$.
- Theo Định lý Thợ, Trường hợp 2:

$$T(n) = O(n^{\log_b a} \log n) = \mathbf{O}(n^2 \log n)$$

c. $T(n) = 3T(n/3) + n \log n$

- $a = 3, b = 3, f(n) = n \log n$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_3 3} = n$ và $n \log n$.
- Ta có $f(n)$ lớn hơn $n^{\log_b a}$ một yếu tố $\log n$.
- Đây là một biến thể của Trường hợp 2 (khi $f(n) = \Theta(n^{\log_b a} \log^k n)$ với $k = 1$):

$$T(n) = \mathbf{O}(n \log^2 n)$$

d. $T(n) = 3T(n/3) + 1$

- $a = 3, b = 3, f(n) = 1$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_3 3} = n$ và 1 .
- Ta có $n^{\log_b a}$ lớn hơn $f(n)$ theo một yếu tố đa thức ($n \in \Omega(1)$).
- Theo Định lý Thợ, Trường hợp 1:

$$T(n) = O(n^{\log_b a}) = \mathbf{O}(n)$$

e. $T(n) = 2T(n/2) + n^n$

- $a = 2, b = 2, f(n) = n^n$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_2 2} = n$ và n^n .
- $f(n)$ lớn hơn $n^{\log_b a}$ rất nhiều. Ta kiểm tra điều kiện chính quy (regularity condition) của Trường hợp 3: $af(n/b) \leq cf(n)$.
- $2 \cdot (n/2)^{n/2} \ll n^n$. Điều kiện này thỏa mãn.
- Theo Định lý Thợ, Trường hợp 3:

$$T(n) = \mathbf{O}(n^n)$$

f. $T(n) = 7T(n/2) + n^2$

- $a = 7, b = 2, f(n) = n^2$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_2 7}$ và n^2 .
- Ta có $\log_2 7 \approx 2.807 > 2$.
- $\implies n^{\log_2 7}$ lớn hơn n^2 theo một yếu tố đa thức.
- Theo Định lý Thợ, Trường hợp 1:

$$T(n) = O(n^{\log_b a}) = \mathbf{O}(n^{\log_2 7})$$

g. $T(n) = 16T(n/4) + n^2$

- $a = 16, b = 4, f(n) = n^2$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_4 16} = n^2$ và n^2 .
- Ta có $n^{\log_b a} = f(n)$.
- Theo Định lý Thợ, Trường hợp 2:

$$T(n) = O(n^{\log_b a} \log n) = \mathbf{O}(n^2 \log n)$$

h. $T(n) = 2T(n/2) - n^2$

- $a = 2, b = 2, f(n) = -n^2$.
- Lưu ý: Định lý Thợ yêu cầu hàm $f(n)$ phải là một hàm không âm. Trong trường hợp này, $f(n) = -n^2$ là hàm âm với $n \geq 1$.
- $\Rightarrow$ Định lý Thợ không áp dụng được trực tiếp.
- (Tuy nhiên, nếu ta thay đổi giả thiết và xét $T(n) = 2T(n/2) + g(n)$ với $g(n) = n^2$, thì $T(n) = \mathbf{O}(n^2)$ theo Trường hợp 3.)

i. $T(n) = 8T(n/2) + n^3$

- $a = 8, b = 2, f(n) = n^3$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_2 8} = n^3$ và n^3 .
- Ta có $n^{\log_b a} = f(n)$.
- Theo Định lý Thợ, Trường hợp 2:

$$T(n) = O(n^{\log_b a} \log n) = \mathbf{O}(n^3 \log n)$$

l. $T(n) = 3T(n/3) + \sqrt{n}$

- $a = 3, b = 3, f(n) = \sqrt{n} = n^{1/2}$.
- So sánh $n^{\log_b a}$ và $f(n)$: $n^{\log_3 3} = n^1$ và $n^{1/2}$.
- Ta có $n^{\log_b a}$ lớn hơn $f(n)$ theo một yếu tố đa thức ($n \in \Omega(\sqrt{n})$).
- Theo Định lý Thợ, Trường hợp 1:

$$T(n) = O(n^{\log_b a}) = \mathbf{O}(n)$$

Bài tập 8:

(a) $T(n) = 2T(n/2) + n^2$

- Hệ số: $a = 2, b = 2$. Độ sâu $h = \log_2 n$.
- Chi phí tại cấp k : $C_k = \text{Số nút} \times \text{Chi phí mỗi nút} = 2^k \cdot \left(\frac{n}{2^k}\right)^2 = 2^k \cdot \frac{n^2}{4^k} = \frac{n^2}{2^k}$.
- Tổng chi phí:

$$T(n) = \sum_{k=0}^{\log_2 n - 1} C_k + \text{Chi phí cơ sở}$$

$$T(n) \approx \sum_{k=0}^{\infty} \frac{n^2}{2^k}$$

$$T(n) = n^2 \sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k$$

$$T(n) = n^2 \cdot \frac{1}{1 - 1/2} = 2n^2$$

- Kết quả: $T(n) = \mathbf{O}(n^2)$

(b) $T(n) = T(n/2) + 1$

- Hệ số: $a = 1, b = 2$. Độ sâu $h = \log_2 n$.
- Chi phí tại cấp k : $C_k = 1^k \cdot 1 = 1$.
- Tổng chi phí:

$$T(n) = \sum_{k=0}^{\log_2 n - 1} 1 + T(1)$$

$$T(n) = \log_2 n \cdot 1 + O(1)$$

- **Kết quả:** $T(n) = O(\log n)$

(c) $T(n) = 2T(n-1) + 1$

- Hệ số: $a = 2$. Kích thước giảm tuyến tính. Độ sâu $h = n - 1$.
- Chi phí tại cấp k : $C_k = \text{Số nút} \times \text{Chi phí mỗi nút} = 2^k \cdot 1 = 2^k$.
- Tổng chi phí:

$$T(n) = \sum_{k=0}^{n-1} C_k$$

$$T(n) = \sum_{k=0}^{n-1} 2^k$$

$$T(n) = \frac{2^{(n-1)+1} - 1}{2 - 1} = 2^n - 1$$

- **Kết quả:** $T(n) = O(2^n)$

(d) $T(n) = 2T(n/2) + n \log n$

- Hệ số: $a = 2, b = 2$. Độ sâu $h = \log_2 n$.
- Chi phí tại cấp k :

$$C_k = 2^k \cdot \left(\frac{n}{2^k} \log \left(\frac{n}{2^k} \right) \right)$$

$$C_k = n (\log n - k) = n \log n - kn$$

- Tổng chi phí:

$$T(n) = \sum_{k=0}^{\log n - 1} C_k + O(n)$$

$$T(n) = \sum_{k=0}^{\log n - 1} (n \log n - kn) + O(n)$$

$$T(n) = n \log n \sum_{k=0}^{\log n - 1} 1 - n \sum_{k=0}^{\log n - 1} k + O(n)$$

$$T(n) = n \log n (\log n) - n \left(\frac{(\log n - 1) \log n}{2} \right) + O(n)$$

$$T(n) = n(\log n)^2 - \frac{1}{2}n(\log n)^2 + \frac{1}{2}n \log n + O(n)$$

$$T(n) = \frac{1}{2}n(\log n)^2 + O(n \log n)$$

- **Kết quả:** $T(n) = O(n \log^2 n)$

Bài tập 10:

1. Tính Lũy thừa

Thuật toán đệ quy cơ bản tính a^n : $Power(a, n) = a \cdot Power(a, n - 1)$ nếu $n > 0$, và 1 nếu $n = 0$.

- **Hệ thức Truy hồi** ($T(n)$ là thời gian tính a^n):

$$\begin{cases} T(n) = T(n - 1) + c & \text{nếu } n > 0 \\ T(0) = c_0 \end{cases}$$

- **Giải:** Sử dụng phương pháp lặp.

$$T(n) = T(n - 1) + c = T(n - 2) + 2c = \dots = T(0) + nc = c_0 + cn$$

- **Kết quả:** $T(n) = O(n)$ (Phù hợp với yêu cầu)

2. Tìm kiếm Tuyến tính

Thuật toán tìm kiếm đệ quy trên đoạn $A[i..j]$ có chiều dài $L = j - i + 1$: kiểm tra $A[i]$, nếu không tìm thấy thì gọi đệ quy trên $A[i + 1..j]$.

- **Hệ thức Truy hồi** ($T(L)$ là thời gian tìm kiếm trên đoạn dài L):

$$\begin{cases} T(L) = T(L - 1) + c & \text{nếu } L > 1 \\ T(1) = c_0 \end{cases}$$

- **Giải:** Tương tự như bài toán lũy thừa, hệ thức này giải ra một hàm tuyến tính.

$$T(L) = c_0 + cL$$

- **Kết quả:** $T(n) = O(n)$ (với n là chiều dài mảng, Phù hợp với yêu cầu)

3. Tìm kiếm Nhị phân

Thuật toán tìm kiếm đệ quy: so sánh phần tử giữa $A[mid]$, sau đó gọi đệ quy trên một nửa mảng (trái hoặc phải).

- **Hệ thức Truy hồi** ($T(n)$ là thời gian tìm kiếm trên mảng dài n):

$$\begin{cases} T(n) = T(n/2) + c & \text{nếu } n > 1 \\ T(1) = c_0 \end{cases}$$

- **Giải:** Sử dụng phương pháp lặp hoặc Định lý Thợ (Trường hợp 2, với $a = 1, b = 2, f(n) = c \in O(n^0)$ và $n^{\log_2 1} = n^0$):

$$T(n) = c_0 + c \log_2 n$$

- **Kết quả:** $T(n) = O(\log n)$ (Phù hợp với yêu cầu)

4. Sắp xếp Nổi bọt

Phiên bản đệ quy: Sắp xếp phần tử lớn nhất về cuối mảng (mất $O(n)$ thời gian), sau đó gọi đệ quy để sắp xếp $n - 1$ phần tử còn lại.

- **Hệ thức Truy hồi** ($T(n)$ là thời gian sắp xếp n phần tử):

$$\begin{cases} T(n) = T(n - 1) + cn & \text{nếu } n > 1 \\ T(1) = c_0 \end{cases}$$

- **Giải:** Sử dụng phương pháp lặp.

$$T(n) = T(n - 1) + cn$$

$$T(n) = T(n - 2) + c(n - 1) + cn$$

$$T(n) = T(1) + c(2 + 3 + \dots + n)$$

$$T(n) = c_0 + c \sum_{i=2}^n i \approx c \sum_{i=1}^n i = c \frac{n(n+1)}{2}$$

- **Kết quả:** $T(n) = O(n^2)$ (Phù hợp với yêu cầu)

5. Sắp xếp Chèn

Phiên bản đệ quy: Gọi đệ quy để sắp xếp $n - 1$ phần tử đầu tiên, sau đó chèn phần tử thứ n vào đúng vị trí trong mảng đã sắp xếp (mất $O(n)$ thời gian).

- **Hệ thức Truy hồi** ($T(n)$ là thời gian sắp xếp n phần tử):

$$\begin{cases} T(n) = T(n-1) + cn & \text{nếu } n > 1 \\ T(1) = c_0 \end{cases}$$

- **Giải:** Hệ thức này giống hệ Sắp xếp Nổi bọt.

$$T(n) = O(n^2)$$

- **Kết quả:** $T(n) = O(n^2)$ (Phù hợp với yêu cầu)

6. Sắp xếp Trộn

Thuật toán đệ quy: Chia mảng thành hai nửa, gọi đệ quy sắp xếp từng nửa, sau đó trộn hai nửa đã sắp xếp lại (mất $O(n)$ thời gian).

- **Hệ thức Truy hồi** ($T(n)$ là thời gian sắp xếp n phần tử):

$$\begin{cases} T(n) = 2T(n/2) + cn & \text{nếu } n > 1 \\ T(1) = c_0 \end{cases}$$

- **Giải:** Sử dụng Định lý Thợ (Trường hợp 2, với $a = 2, b = 2, f(n) = cn \in O(n^1)$ và $n^{\log_2 2} = n^1$):

$$T(n) = O(n^{\log_2 2} \log n) = O(n \log n)$$

- **Kết quả:** $T(n) = O(n \log n)$ (Phù hợp với yêu cầu)